

252-0027

Einführung in die Programmierung

Übungen

Woche 8: Klassen, Loop-Invarianten

Joran Bühler

Departement Informatik

ETH Zürich

Klassen

Klassen und Objekte – Allgemein

- Eine *Klasse* ist die Vorlage (Schablone, Bauplan), die alle *Objekte* dieses (Klassen-)Typs beschreibt
 - Objekte werden gemäss dieser Vorlage erschaffen
 - *Attribute* beschreiben den möglichen Zustandsraum
 - *Methoden* (später) beschreiben Funktionalität
- Objekte sind **Instanzen** (Exemplare) einer Klasse

```
// Vorlage
public class Name {
    type1 attrName1;
    type2 attrName2;
    ...
}
```

```
// Instanzen
Name n1 = new Name();
Name n2 = new Name();
...
```

Klassen und Objekte – Allgemein

- Eine *Klasse* ist die Vorlage (Schablone, Bauplan), die alle *Objekte* dieses (Klassen-)Typs beschreibt
- Objekte müssen erzeugt werden, bevor mit ihnen gearbeitet werden kann
 - Üblicherweise mittels `new ClassName(...)`
 - Alternativ mittels Literalen (Strings, Arrays)
 - `String name = "Ravi";`
 - `double[] grades = {4.0, 5.5, 5.0, 3.75};`

Probleme Lösen: Zeitspanne

In dieser Aufgabe wollen wir eine Klasse Zeitspanne erstellen. Objekte dieser Klasse speichern eine Zeitdauer (z.B. 3 Stunden, 45 Minuten) und geben diese auf Anfrage zurück.

1. Erstellen Sie die Klasse Zeitspanne. Diese Klasse erstellt Objekte welche eine Zeitspanne darstellen (in Stunden und Minuten, Sekunden sind nicht verlangt).
2. Erstellen Sie passende Methoden für diese Klasse. Wir sollten die Zeitspanne verändern können (setzen auf einen beliebigen (sinnvollen) Wert, addieren und subtrahieren), sie in Minuten zurückgeben können und drucken können.
3. Schreiben Sie nun eine Methode 'isMoreThan()', die eine zweite Zeitspanne als Argument nimmt und es ermöglicht zwei Zeitspannen zu vergleichen.

```
public class Zeitspanne {  
    public int minutes;  
    public int hours;  
}
```

1. Erstellen Sie die Klasse Zeitspanne. Diese Klasse erstellt Objekte welche eine Zeitspanne darstellen (in Stunden und Minuten, Sekunden sind nicht verlangt).

```
public class Zeitspanne {  
    public int minutes;  
    public int hours;  
}
```

Ist nicht notwendig...

94 Minuten $\simeq$ 1 Stunde und 34 Minuten

1. Erstellen Sie die Klasse Zeitspanne. Diese Klasse erstellt Objekte welche eine Zeitspanne darstellen (in Stunden und Minuten, Sekunden sind nicht verlangt).

(...)

```
public int getFullHours() {  
    return this.minutes / 60;  
}
```

```
// Setter Methoden  
public void setMinutes(int minutes) {  
    this.minutes = minutes;  
}
```



Soll ein Benutzer (indirect) Zugriff
auf das minutes Attribut haben?

Hier: Ja

Dies ist aber oft nicht der Fall!

}

2. Erstellen Sie passende Methoden für diese Klasse. Wir sollten die Zeitspanne verändern können (setzen auf einen beliebigen (sinnvollen) Wert, addieren und subtrahieren), sie in Minuten zurückgeben können und drucken können. Wenn der user unpassende

(...)

```
// Setter Methoden
```

```
public void setMinutes(int minutes) {  
    this.minutes = minutes;  
}
```

```
public void setTime(int hours, int minutes) {  
    this.minutes = hours * 60 + minutes;  
}
```

```
}
```

Methoden für **jede** Klasse:

☐ Konstruktoren

☐ toString Methode

Später mehr...

2. Erstellen Sie passende Methoden für diese Klasse. Wir sollten die Zeitspanne verändern können (setzen auf einen beliebigen (sinnvollen) Wert, addieren und subtrahieren), sie in Minuten zurückgeben können und drucken können. Wenn der user unpassende


```
public class Zeitspanne {  
    private int minutes;  
  
    (...)  
  
    public Zeitspanne(int hours, int minutes) {  
        this.minutes = minutes + hours * 60;  
    }  
  
    public Zeitspanne(int minutes) {  
        this.minutes = minutes;  
    }  
  
    public Zeitspanne(Zeitspanne other) {  
        this.minutes = other.minutes;  
    }  
  
    (...)  
}
```

Konstruktoren:

- ❑ Default Konstruktor
- ❑ Parametrisierte Konstruktor(en)
- ❑ Copy Konstruktor

Wieso?

Wir erstellen ein Objekt mit einem anderen Objekt als Vorlage.

```
public class Zeitspanne {  
    private int minutes;  
  
    (...)  
    public Zeitspanne(int hours, int minutes) {  
        this.minutes = minutes + hours * 60;  
    }  
  
    public Zeitspanne(int minutes) {  
        this.minutes = minutes;  
    }  
  
    public Zeitspanne(Zeitspanne other) {  
        this.minutes = other.minutes;  
    }  
  
    (...)  
}
```



Sehr repetitiv...


```
public class Zeitspanne {  
    private int minutes;  
    public Zeitspanne(int minutes) {  
        this.minutes = minutes;  
    }  
    public Zeitspanne() {  
        this(0);  
    }  
    public Zeitspanne(int hours, int minutes) {  
        this(minutes + hours * 60);  
    }  
    public Zeitspanne(Zeitspanne other) {  
        this(other.minutes);  
    }  
    (...)  
}
```



Einen parametrisierten Konstruktor.
Alle anderen Konstruktoren rufen diesen auf.


```
public class Zeitspanne {  
    (...)  
    public void setTime(int hours, int minutes) {  
        this.minutes = hours * 60 + minutes;  
    }  
  
    public String toString() {  
        int hours = this.minutes / 60;  
        int minutes = this.minutes % 60;  
  
        return hours + ":" + minutes;  
    }  
}
```

Methoden für **jede** Klasse:

❑ Konstruktoren

❑ toString Methode

```
public class Zeitspanne {  
    (...)  
    public void addTime(int hours, int minutes) {  
        this.minutes += toMinutes(hours, minutes);  
    }  
  
    public void subTime(int hours, int minutes) {  
        this.minutes -= toMinutes(hours, minutes);  
    }  
}
```

2. Erstellen Sie passende Methoden für diese Klasse. Wir sollten die Zeitspanne verändern können (setzen auf einen beliebigen (sinnvollen) Wert, **addieren und subtrahieren**), sie in Minuten zurückgeben können und drucken können. Wenn der user unpassende

```
public class Zeitspanne {  
    (...)  
    public void addTime(int hours, int minutes) {  
        this.minutes += toMinutes(hours, minutes);  
    }  
  
    public void subTime(int hours, int minutes) {  
        this.minutes -= toMinutes(hours, minutes);  
    }  
}
```

Was wenn die minutes negativ werden würden?

Jetzt: boolean Rückgabewert, der sagt, ob die Operation erfolgreich war.

Später: Exceptions / Errors

2. Erstellen Sie passende Methoden für diese Klasse. Wir sollten die Zeitspanne verändern können (setzen auf einen beliebigen **(sinnvollen)** Wert, addieren und subtrahieren), sie in Minuten zurückgeben können und drucken können. Wenn der user unpassende

```
public class Zeitspanne {  
    (...)  
    public void addTime(int hours, int minutes) {  
        this.minutes += toMinutes(hours, minutes);  
    }  
    public boolean subTime(int hours, int minutes) {  
        int minutesToSub = toMinutes(hours, minutes);  
        if(this.minutes >= minutesToSub) {  
            this.minutes -= minutesToSub;  
            return true;  
        }  
        return false;  
    }  
}
```

Jetzt: boolean Rückgabewert,
der sagt, ob die Operation
erfolgreich war.

Später: Exceptions / Errors

2. Erstellen Sie passende Methoden für diese Klasse. Wir sollten die Zeitspanne verändern können (setzen auf einen beliebigen **(sinnvollen)** Wert, addieren und subtrahieren), sie in Minuten zurückgeben können und drucken können. Wenn der user unpassende

```
public class Zeitspanne {  
    (...)  
    public boolean isMoreThan(Zeitspanne other) {  
        return this.getTimeInMin() > other.getTimeInMin();  
    }  
  
}
```

3. Schreiben Sie nun eine Methode 'isMoreThan()', die eine zweite Zeitspanne als Argument nimmt und es ermöglicht zwei Zeitspannen zu vergleichen.

Loop - Invarianten


```
public int compute(int a, int b) {  
    // Precondition:  a >= 0  
    int x;  
    int res;  
  
    x = a;  
    res = b;  
    // Loop-Invariante: ??  
    while (x > 0) {  
        x = x - 1;  
        res = res + 1;  
    }  
    // Postcondition:  res == a + b  
    return res;  
}
```

Loop-Invarianten

1. Die Loop-Invariante ist **vor der Schleife** erfüllt.
2. Die Loop-Invariante ist **nach jeder Ausführung** des while – body erfüllt.
3. Die Loop-Invariante ist **nach der Schleife** erfüllt.

```

public int compute(int a, int b) {
    // Precondition:  a >= 0
    int x;
    int res;

    x = a;
    res = b;
    // Loop-Invariante: ??
    while (x > 0) {
        x = x - 1;
        res = res + 1;
    }
    // Postcondition:  res == a + b
    return res;
}

```

Loop-Invarianten

1. Die Loop-Invariante ist **vor der Schleife** erfüllt.
2. Die Loop-Invariante ist **nach jeder Ausführung** des while – body erfüllt.
3. Die Loop-Invariante ist **nach der Schleife** erfüllt.

Wir wollen das folgende Hoare Triple beweisen:

```

{ Precondition }
  while ( Condition ) { Body };
{ Postcondition }

```

```

public int compute(int a, int b) {
    // Precondition:  a >= 0

    int x;
    int res;

    x = a;
    res = b;
    // Loop-Invariante: ??
    while (x > 0) {
        x = x - 1;
        res = res + 1;
    }
    // Postcondition:  res == a + b
    return res;
}

```

Wir wollen das folgende Hoare Triple beweisen:

```

{ Precondition }
  while ( Condition ) { Body };
{ Postcondition }

```

Dies können wir tun, falls eine Invariante existiert, für welche folgendes gilt:

1. $\text{Precondition} \Rightarrow \text{Invariante}$
2. $\{ \text{Condition} \wedge \text{Invariante} \}$
 $\text{Body};$
 $\{ \text{Invariante} \}$ ist ein valides Tripel.
3. $\neg \text{Condition} \wedge \text{Invariante} \Rightarrow \text{Postcondition}$

}

Herleitungsbeispiel mit Tabelle

[illegible]

}

Herleitungsbeispiel mit Tabelle

Iteration	res	x
0	b	a

```

public int compute(int a, int b) {
    // Precondition:  a >= 0
    int x;
    int res;

    x = a;
    res = b;
    // Loop-Invariante: ??
    while (x > 0) {
        x = x - 1;
        res = res + 1;
    }
    // Postcondition:  res == a + b
    return res;
}

```

Herleitungsbeispiel mit Tabelle

Iteration	res	x
0	b	a
1	b + 1	a - 1

```

public int compute(int a, int b) {
    // Precondition:  a >= 0
    int x;
    int res;

    x = a;
    res = b;
    // Loop-Invariante: ??
    while (x > 0) {
        x = x - 1;
        res = res + 1;
    }
    // Postcondition:  res == a + b
    return res;
}

```

Herleitungsbeispiel mit Tabelle

Iteration	res	x
0	b	a
1	b + 1	a - 1
2	b + 2	a - 2

```

public int compute(int a, int b) {
    // Precondition:  a >= 0
    int x;
    int res;

    x = a;
    res = b;
    // Loop-Invariante: ??
    while (x > 0) {
        x = x - 1;
        res = res + 1;
    }
    // Postcondition:  res == a + b
    return res;
}

```

Herleitungsbeispiel mit Tabelle

Iteration	res	x
0	b	a
1	b + 1	a - 1
2	b + 2	a - 2
3	b + 3	a - 3


```

public int compute(int a, int b) {
    // Precondition:  a >= 0
    int x;
    int res;

    x = a;
    res = b;
    // Loop-Invariante: ??
    while (x > 0) {
        x = x - 1;
        res = res + 1;
    }
    // Postcondition:  res == a + b
    return res;
}

```

Herleitungsbeispiel mit Tabelle

Iteration	res	x
0	b	a
1	b + 1	a - 1
2	b + 2	a - 2
3	b + 3	a - 3
...	...	...

```

public int compute(int a, int b) {
    // Precondition:  a >= 0
    int x;
    int res;

    x = a;
    res = b;
    // Loop-Invariante: ??
    while (x > 0) {
        x = x - 1;
        res = res + 1;
    }
    // Postcondition:  res == a + b
    return res;
}

```

Herleitungsbeispiel mit Tabelle

Iteration	res	x
0	b	a
1	b + 1	a - 1
2	b + 2	a - 2
3	b + 3	a - 3
...	...	...
i	b + i	a - i

```

public int compute(int a, int b) {
    // Precondition:  a >= 0
    int x;
    int res;

    x = a;
    res = b;
    // Loop-Invariante: ??
    while (x > 0) {
        x = x - 1;
        res = res + 1;
    }
    // Postcondition:  res == a + b
    return res;
}

```

Herleitungsbeispiel mit Tabelle

Iteration	res	x
0	b	a
1	b + 1	a - 1
2	b + 2	a - 2
3	b + 3	a - 3
...	...	...
i	b + i	a - i

LI: $\text{res} == b + a - x$

```

public int compute(int a, int b) {
    // Precondition:  a >= 0
    int x;
    int res;

    x = a;
    res = b;
    // Loop-Invariante: ??
    while (x > 0) {
        x = x - 1;
        res = res + 1;
    }
    // Postcondition:  res == a + b
    return res;
}

```

1. $\text{Precondition} \Rightarrow \text{Invariante}$
2. $\{ \text{Condition} \wedge \text{Invariante} \}$
 $\text{Body};$
 $\{ \text{Invariante} \}$ ist ein valides Tripel.
3. $\neg \text{Condition} \wedge \text{Invariante} \Rightarrow$
 Postcondition

LI: $\text{res} == \text{b} + \text{a} - \text{x}$

Reicht das?

```

public int compute(int a, int b) {
    // Precondition:  a >= 0

    int x;
    int res;

    x = a;
    res = b;
    // Loop-Invariante: ??
    while (x > 0) {
        x = x - 1;
        res = res + 1;
    }
    // Postcondition:  res == a + b
    return res;
}

```

1. Precondition $\Rightarrow$ Invariante ✓

2. { Condition $\wedge$ Invariante }
 Body;
 { Invariante } ist ein valides Tripel. ✓

3. $\neg$ Condition $\wedge$ Invariante $\Rightarrow$
 Postcondition ✗

LI: res == b + a - x

Reicht das?

```

public int compute(int a, int b) {
    // Precondition:  a >= 0

    int x;
    int res;

    x = a;
    res = b;
    // Loop-Invariante: ??
    while (x > 0) {
        x = x - 1;
        res = res + 1;
    }
    // Postcondition:  res == a + b
    return res;
}

```

1. Precondition $\Rightarrow$ Invariante ✓

2. { Condition $\wedge$ Invariante }
 Body;
 { Invariante } ist ein valides Tripel. ✓

3. $\neg$ Condition $\wedge$ Invariante $\Rightarrow$
 Postcondition ✓

res == b + a - x && x >= 0

Beispiel 2


```

public static int factorial(int n) {
    // Precondition: n >= 0
    int counter;
    int result;

    counter = n;
    result = 1;
    // Loop-Invariante: ??
    while (counter > 0) {
        result = result * counter;
        counter = counter - 1;
    }

    // Postcondition: result = n!
    return result;
}

```

Herleitungsbeispiel mit Tabelle

Iteration	result	counter
0	1	n
1	n	n - 1

```

public static int factorial(int n) {
    // Precondition: n >= 0
    int counter;
    int result;

    counter = n;
    result = 1;
    // Loop-Invariante: ??
    while (counter > 0) {
        result = result * counter;
        counter = counter - 1;
    }

    // Postcondition: result = n!
    return result;
}

```

Herleitungsbeispiel mit Tabelle

Iteration	result	counter
0	1	n
1	n	n - 1
2	n * (n - 1)	n - 2

```

public static int factorial(int n) {
    // Precondition: n >= 0
    int counter;
    int result;

    counter = n;
    result = 1;
    // Loop-Invariante: ??
    while (counter > 0) {
        result = result * counter;
        counter = counter - 1;
    }

    // Postcondition: result = n!
    return result;
}

```

Herleitungsbeispiel mit Tabelle

Iteration	result	counter
0	1	n
1	n	n - 1
2	$n * (n - 1)$	n - 2
3	...	n - 3

```

public static int factorial(int n) {
    // Precondition: n >= 0
    int counter;
    int result;

    counter = n;
    result = 1;
    // Loop-Invariante: ??
    while (counter > 0) {
        result = result * counter;
        counter = counter - 1;
    }

    // Postcondition: result = n!
    return result;
}

```

Herleitungsbeispiel mit Tabelle

Iteration	result	counter
0	1	n
1	n	n - 1
2	$n * (n - 1)$	n - 2
3	...	n - 3
...	...	...

```

public static int factorial(int n) {
    // Precondition: n >= 0
    int counter;
    int result;

    counter = n;
    result = 1;
    // Loop-Invariante: ??
    while (counter > 0) {
        result = result * counter;
        counter = counter - 1;
    }

    // Postcondition: result = n!
    return result;
}

```

Herleitungsbeispiel mit Tabelle

Iteration	result	counter
0	1	n
1	n	n - 1
2	$n * (n - 1)$	n - 2
3	...	n - 3
...	...	...
i	$n! / (n - i)!$	n - i

```

public static int factorial(int n) {
    // Precondition: n >= 0
    int counter;
    int result;

    counter = n;
    result = 1;
    // Loop-Invariante: ??
    while (counter > 0) {
        result = result * counter;
        counter = counter - 1;
    }

    // Postcondition: result = n!
    return result;
}

```

Herleitungsbeispiel mit Tabelle

Iteration	result	counter
0	1	n
1	n	n - 1
2	$n * (n - 1)$	n - 2
3	...	n - 3
...	...	...
i	$n! / (n - i)!$	n - i

LI: $\text{result} == n! / \text{counter}!$

```

public static int factorial(int n) {
    // Precondition: n >= 0

    int counter;
    int result;

    counter = n;
    result = 1;
    // Loop-Invariante: ??
    while (counter > 0) {
        result = result * counter;
        counter = counter - 1;
    }

    // Postcondition: result = n!
    return result;
}

```

1. Precondition $\Rightarrow$ Invariante ✓

2. { Condition $\wedge$ Invariante }
 Body;
 { Invariante } ist ein valides Tripel. ✓

3. $\neg$ Condition $\wedge$ Invariante $\Rightarrow$
 Postcondition ✗

LI: result == n! / counter!

Reicht das?


```

public static int factorial(int n) {
    // Precondition: n >= 0
    int counter;
    int result;

    counter = n;
    result = 1;
    // Loop-Invariante: ??
    while (counter > 0) {
        result = result * counter;
        counter = counter - 1;
    }

    // Postcondition: result = n!
    return result;
}

```

1. Precondition $\Rightarrow$ Invariante ✓

2. { Condition $\wedge$ Invariante }
 Body;
 { Invariante } ist ein valides Tripel. ✓

3. $\neg$ Condition $\wedge$ Invariante $\Rightarrow$
 Postcondition ✓

LI: result == n! / counter! &&
 counter >= 0

Rezept für Loop-Invarianten

Es gibt keine perfekten Rezepte!



- Loop-Invarianten richtig lösen können ist eine **Frage der Übung**.
- Das folgende Rezept hat sich in der Vergangenheit bei vielen Studenten als nützlich erwiesen.

```

public int compute(int n){
    // Precondition: n >= 0
    int k = 5;
    int i = 0;
    int result = 1;

    // Loop-Invariante: ??
    while (i < n) {
        result = result * k;
        i = i + 1;
    }
    //Postcondition result = 5^n
    return result;
}

```

1. Loop Condition und Terminierung kombinieren.

$$\{i \leq n\}$$

2. Postcondition und Loop Body kombinieren.

$$\{result == 5^n\} \rightarrow \{result == 5^i\}$$

3. Conditions wegen benutzter Methoden oder mathematischer Formeln.

$$\{k == 5\}$$

$$\{i \leq n \ \&\& \ result == 5^i \ \&\& \ k == 5 \}$$

```

public int compute(int v, int n) {
    // Precondition: n >= 0
    int v = 2;
    int x;
    int tmp;

    x = 1;
    tmp = 1;
    // Loop-Invariante: ??
    while (x <= n) {
        tmp = tmp * v;
        x = x + 1;
    }
    // Postcondition: tmp == 2^n
    return tmp;
}

```

Wir wollen das folgende Hoare Triple beweisen:

```

{ Precondition }
  while ( Condition ) { Body };
{ Postcondition }

```

Dies können wir tun, falls eine Invariante existiert, für welche folgendes gilt:

1. $\text{Precondition} \Rightarrow \text{Invariante}$
2. $\{ \text{Condition} \wedge \text{Invariante} \}$
 $\text{Body};$
 $\{ \text{Invariante} \}$ ist ein valides Tripel.
3. $\neg \text{Condition} \wedge \text{Invariante} \Rightarrow \text{Postcondition}$

```

public int compute(int v, int n) {
    // Precondition: n >= 0
    int v = 2;
    int x;
    int tmp;

    x = 1;
    tmp = 1;
    // Loop-Invariante: ??
    while (x <= n) {
        tmp = tmp * v;
        x = x + 1;
    }
    // Postcondition: tmp == 2^n
    return tmp;
}

```

1. $\text{Precondition} \Rightarrow \text{Invariante}$
2. $\{ \text{Condition} \wedge \text{Invariante} \}$
 $\text{Body};$
 $\{ \text{Invariante} \}$ ist ein valides Tripel.
3. $\neg \text{Condition} \wedge \text{Invariante} \Rightarrow$
 Postcondition

LI: $v == 2 \ \&\& \ tmp == 2^{(x - 1)}$
 $\&\& \ x \leq n + 1 \ \&\& \ x \geq 1$

Erklärungsvideo:

- <https://eprog23.wixstudio.io/hs23/videos>

- Meine TA
- Extrem gute Erklärungen, auch zu Invarianten



Kahoot

<https://create.kahoot.it/share/u8-eprog-25-klassen-basics-loop-invarianten/0941ad27-5d84-4aff-884f-4e0f52d6b607>

Aufgabe 4: Minesweeper (Bonus)

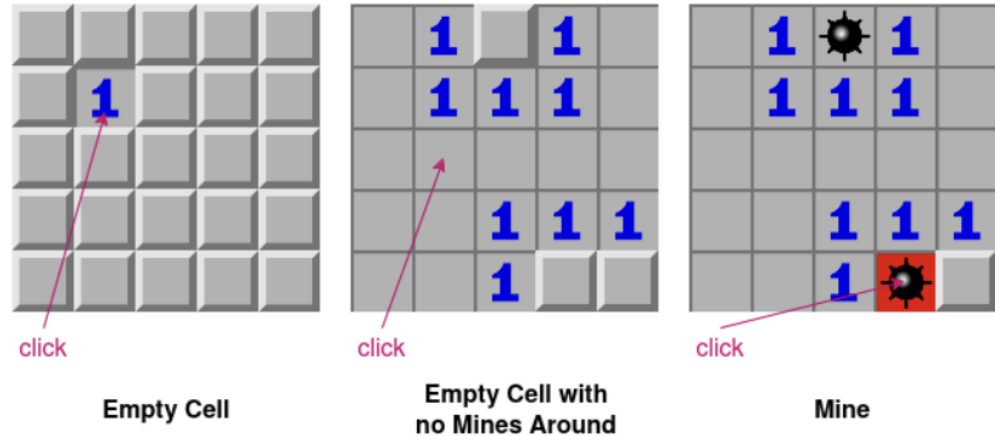


Abbildung 1: Spielbretter nach dem ersten, zweiten und dritten Klick von links nach rechts.

Achtung: Diese Aufgabe gibt Bonuspunkte (siehe "Leistungskontrolle" im www.vvz.ethz.ch). Die Aufgabe muss eigenhändig und alleine gelöst werden. Die Abgabe erfolgt wie gewohnt per Push in Ihr Git-Repository auf dem ETH-Server. Verbindlich ist der letzte Push vor dem Abgabetermin. Auch wenn Sie vor der Deadline committen, aber nach der Deadline pushen, gilt dies als eine zu späte Abgabe. Bitte lesen Sie zusätzlich [die allgemeinen Regeln](#).